VINUNIVERSITY

CI/CD cho hệ thống AI

AICB-P2T2 · Ngày 21 · Chương 5 — Vận hành

Giảng viên Nguyễn Hải Dương

VinUniversity · Phase 2 · Track 2 · Tuần 5

HÃY SUY NGHĨ...

“Giả sử một nhóm cập nhật lời nhắc (prompt) của agent vào chiều thứ Sáu. Đến sáng thứ Hai, tỉ lệ lời gọi công cụ (tool call) thành công giảm từ 92% xuống 71%, nhưng không cảnh báo (alert) nào được gửi đi. Với hệ thống AI, một lần thay đổi nhỏ phải đi qua những bước kiểm tra nào trước khi đến người dùng?”

Giữ câu hỏi này trong đầu khi học bài hôm nay

Nội dung bài học

1	Vì sao AI cần tích hợp và triển khai liên tục (CI/CD) riêng	15 phút
2	MLflow: theo dõi thí nghiệm, kho mô hình, đánh giá mô hình ngôn ngữ lớn (LLM)	30 phút
3	DVC: quản lý phiên bản dữ liệu và chuỗi xử lý (pipeline)	25 phút
4	GitHub Actions: CI và cổng đánh giá (gate)	30 phút
5	CD: chiến lược triển khai (deployment) và GitOps	20 phút
6	Kim tự tháp kiểm thử (testing) cho hệ thống AI	15 phút
7	Phục vụ mô hình và kiểm thử A/B	15 phút
8	Chuỗi cung ứng mô hình và bảo mật CI/CD	10 phút
9	Trình diễn (demo), tổng kết và bài tập	20 phút

Mục tiêu bài học

Sau buổi học, học viên có thể:

1. Thiết lập máy chủ (server) theo dõi MLflow, ghi lại thí nghiệm, đăng ký mô hình bằng bí danh (alias) và đánh giá ứng dụng LLM bằng `mlflow.genai.evaluate`.
2. Quản lý phiên bản dữ liệu bằng DVC và dựng chuỗi xử lý có thể tái lập (reproducible) với `dvc.yaml`.
3. Viết luồng công việc (workflow) GitHub Actions gồm kiểm thử, kiểm tra dữ liệu, huấn luyện, cổng đánh giá và triển khai.
4. Chọn chiến lược triển khai (canary, blue/green, shadow) và dựng điều kiện tự động hoàn tác.
5. Nhận diện rủi ro chuỗi cung ứng của mô hình và cách giảm: định dạng safetensors, ký số, chứng thực nguồn gốc.

Sản phẩm cuối ngày

Máy chủ MLflow + tập dữ liệu (dataset) có phiên bản DVC + luồng GitHub Actions tự động kiểm thử và triển khai

MLflow
Từ 3 thí nghiệm trở lên, đủ tham số, chỉ số và sản phẩm đầu ra (artifact).

DVC
Chuỗi xử lý 3 giai đoạn (prepare, train, evaluate) chạy thành công bằng `dvc repro`.

GitHub Actions
Luồng công việc đạt: kiểm thử, huấn luyện, cổng đánh giá, triển khai trên kho mã thực.

Kho mô hình
Mô hình được đăng ký và gán bí danh `challenger` sau khi qua cổng đánh giá.

Vì sao AI cần CI/CD riêng

01

Mã chỉ là một phần nhỏ của hệ thống AI; dữ liệu, mô hình và lời nhắc cũng thay đổi và cũng gây lỗi

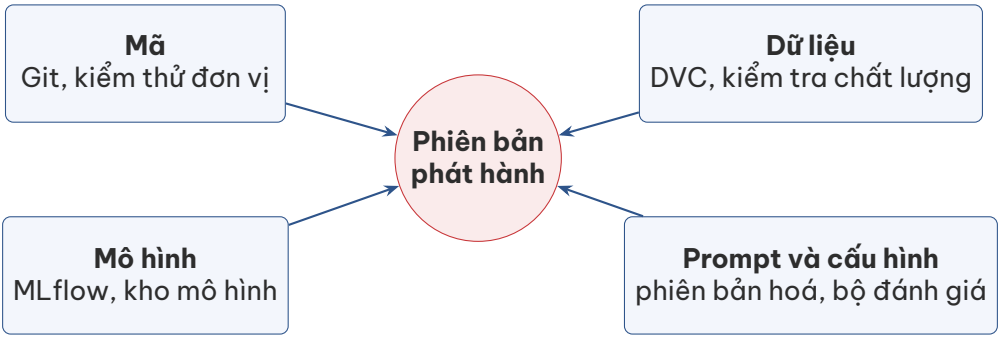
Mã ML chỉ là một phần nhỏ của hệ thống



Phần mã mô hình chiếm một ô nhỏ; hạ tầng bao quanh lớn và phức tạp hơn nhiều, và mỗi ô đều cần được kiểm thử và triển khai tự động.

Nguồn: Vẽ lại theo Sculley và cộng sự, Hidden Technical Debt in Machine Learning Systems, NIPS 2015, Hình 1

Bốn trục thay đổi của một hệ thống AI



Chỉ cần một trục đổi, kết quả đổi theo: cùng một mã, dữ liệu mới vẫn có thể làm mô hình xấu đi. CI/CD cho AI phải kích hoạt khi **bất kỳ** trục nào thay đổi.

CI/CD cho AI khác phần mềm thông thường

Khía cạnh	Phần mềm truyền thống	Hệ thống AI
Sản phẩm đầu ra (artifact)	Tệp nhị phân, image	Trọng số (weight) của mô hình, lời nhắc, cấu hình, siêu dữ liệu
Kiểm thử	Kiểm thử đơn vị, tích hợp	Thêm: đánh giá mô hình, kiểm tra dữ liệu, bộ đánh giá LLM
Phiên bản	Git cho mã	Git cho mã, DVC cho dữ liệu, kho mô hình cho mô hình
Kết quả	Xác định: cùng đầu vào, cùng đầu ra	Không xác định: cần ngưỡng và lặp lại nhiều lần
Điều kiện hoàn tác	Tỉ lệ lỗi tăng	Thêm: độ chính xác giảm, thiên lệch, chi phí token tăng
Kích hoạt chuỗi xử lý	Mã thay đổi	Mã, dữ liệu, lời nhắc thay đổi; lịch; dữ liệu trôi (drift)
Thời gian chạy	Vài phút	Vài phút đến vài giờ khi có huấn luyện

Ba mức trưởng thành MLOps

Mức 0 · Thủ công

Mọi bước làm tay: phân tích dữ liệu, huấn luyện, kiểm định, bàn giao cho nhóm vận hành.

Mức 1 · Tự động hoá chuỗi

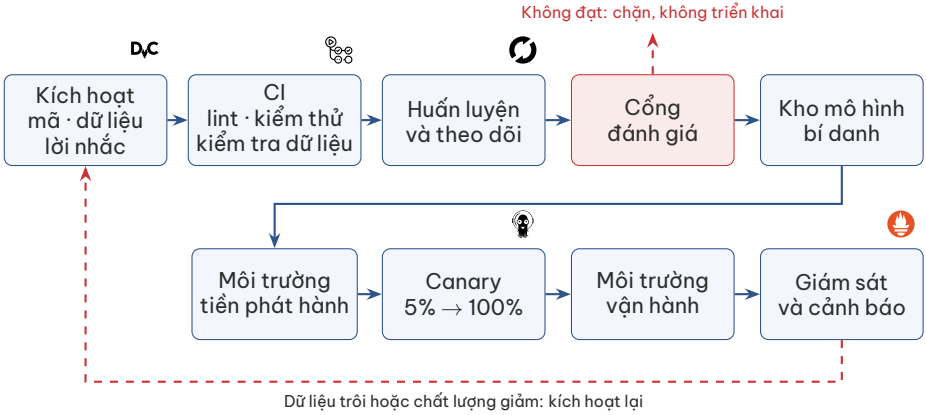
Tự động hoá chuỗi huấn luyện: **huấn luyện liên tục (CT)**, kiểm tra dữ liệu và mô hình tự động, siêu dữ liệu được lưu.

Mức 2 · CI/CD tự động

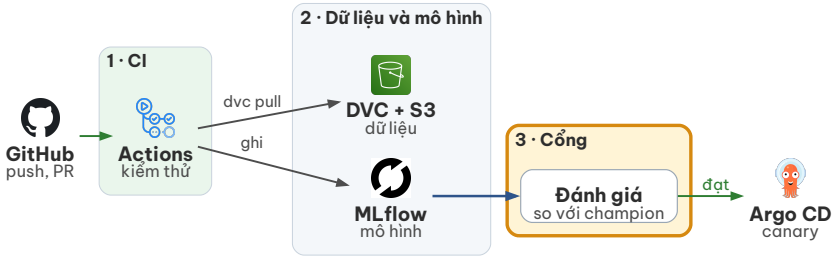
Hệ thống CI/CD tự động: xây dựng, kiểm thử và triển khai chính chuỗi huấn luyện; nhóm thử ý tưởng mới nhanh và an toàn.

Nguồn: Google Cloud Architecture Center, MLOps: Continuous delivery and automation pipelines in machine learning (cập nhật 08/2024)

Chuỗi CI/CD tổng thể cho hệ thống AI



Kiến trúc tham chiếu: công cụ cho từng bước



1. GitHub Actions chạy kiểm thử, lấy dữ liệu bằng `dvc pull` từ S3 **2.** Huấn luyện ghi lần chạy và mô hình vào MLflow
3. Cổng đánh giá so `challenger` với `champion` **4.** Đạt thì Argo CD đồng bộ phiên bản mới và chạy canary
Màu cạnh: xám là dữ liệu, xanh lục là xây dựng và triển khai, xanh dương là truy vấn.

Nguồn: tự vẽ (archdiag), biểu tượng AWS Architecture Icons và logo công cụ

Ba sự cố thật: bộ đánh giá không phát hiện được

Sự cố	Điều đã xảy ra	Bài học cho CI/CD
GPT-4o, 04/2025	Bản cập nhật khiến mô hình phản hồi quá chiều lòng người dùng; được khôi phục sau vài ngày. Bộ đánh giá trước phát hành đạt, không có chỉ số (metric) đo hiện tượng này.	Cổng đánh giá chỉ chặn được điều mà nó đo; cần thêm chỉ số khi biết kiểu lỗi mới.
Anthropic, 08–09/2025	Ba lỗi hạ tầng suy giảm chất lượng đầu ra; lỗi định tuyến (routing) ảnh hưởng tối đa 16% yêu cầu (request) Sonnet 4 ở giờ xấu nhất. Bộ đánh giá không phản ánh suy giảm người dùng báo.	Chạy đánh giá liên tục trên hệ thống thật, không chỉ trước phát hành.
Gemini, 02/2024	Tính năng tạo ảnh người bị tạm dừng sau khi sinh ra kết quả sai lệch.	Kiểm thử chất lượng và an toàn phải gồm cả ca biên.

Chưa có nguồn gốc đáng tin về một ca canary tự phát hiện suy giảm LLM; phần trình diễn dùng kịch bản mô phỏng.

Nguồn: OpenAI (05/2025); Anthropic Engineering, A postmortem of three recent issues (17/09/2025); Google (23/02/2024)

MLflow: thí nghiệm, kho mô hình, đánh giá LLM

Theo dõi thí nghiệm, đăng ký mô hình bằng bí danh, truy vết và đánh giá ứng dụng LLM

02

MLflow 3: bốn nhóm chức năng

MLflow là nền tảng mã nguồn mở quản lý vòng đời ML và ứng dụng GenAI. Bản hiện hành: **3.16** (09/2026).

Tracking	Ghi tham số, chỉ số, sản phẩm đầu ra (artifact) và mã nguồn của mỗi lần chạy (run); so sánh qua giao diện web.
Model Registry	Đăng ký phiên bản mô hình; gán bí danh (alias) và thẻ (tag) thay cho các giai đoạn (stage) đã bị loại bỏ dần.
GenAI	Truy vết (tracing), `mlflow.genai.evaluate`, bộ chấm điểm (scorer) và kho lời nhắc (Prompt Registry).
AI Gateway	Một điểm truy cập cho nhiều nhà cung cấp LLM, có chia lưu lượng.

Nguồn: MLflow release v3.1.0 và v3.16.1 (GitHub); tài liệu MLflow



MLflow 3 thêm thực thể `LoggedModel`: mô hình trở thành đối tượng độc lập, liên kết với lần chạy, vết thực thi, lời nhắc và kết quả đánh giá.

MLflow Tracking: ghi lại một thí nghiệm

```python
import mlflow
from mlflow.models import infer_signature

mlflow.set_experiment("sentiment-v2")
with mlflow.start_run(run_name="lr_0001_ep10"):
    mlflow.log_params({"lr": 0.001, "epochs": 10,
                       "data_version": "v3"})
    for epoch in range(10):
        loss = train_one_epoch(model, loader)
        acc = evaluate(model, val_loader)
        mlflow.log_metrics({"train_loss": loss,
                            "val_accuracy": acc}, step=epoch)
    mlflow.log_artifact("confusion_matrix.png")
    mlflow.sklearn.log_model(
        sk_model=model, name="model",
        signature=infer_signature(X_val, y_pred),
        registered_model_name="sentiment_classifier")
```

Params Siêu tham số và phiên bản dữ liệu; bất biến trong một lần chạy.

Metrics Ghi theo từng bước để vẽ đường cong so sánh.

Artifacts Biểu đồ, tệp cấu hình lưu cùng lần chạy để truy ngược.

Signature Lược đồ đầu vào và đầu ra, cần khi phục vụ mô hình.

Tham số `name=` thay cho `artifact_path=` trong tài liệu MLflow 3.

So sánh lần chạy và truy vết LLM tự động

So sánh lần chạy trên giao diện

- Lọc và sắp xếp lần chạy theo bất kỳ chỉ số nào
- Biểu đồ toạ độ song song: thấy ngay tốc độ học nào cho `loss` thấp nhất
- Biểu đồ phân tán: độ chính xác so với độ trễ (latency)
- So sánh tham số giữa hai lần chạy để tìm nguyên nhân suy giảm
- Xem ma trận nhầm lẫn ngay trên giao diện

```
import mlflow

mlflow.openai.autolog()       # OpenAI
mlflow.langchain.autolog()    # LangChain
# Tu dong ghi vet thuc thi (trace):
#  - prompt, dau vao / dau ra
#  - token (prompt, completion)
#  - do tre tung buoc
#  - ten va phien ban mo hinh
#  - ngu canh truy xuat (RAG)
```

Không cần sửa mã ứng dụng: gọi `autolog()` trước khi chạy. Hỗ trợ OpenAI, LangChain, LlamaIndex, Anthropic. Có từ bản 2.14 (06/2024), là điểm vào của truy vết GenAI trong MLflow 3.

Kho mô hình: từ giai đoạn sang bí danh

Mô hình đã đăng ký: `sentiment_classifier`



Cũ (đã loại bỏ dần từ 2.9, 12/2023)

```
models:/name/Production
transition_model_version_stage
```

Mới

```
models:/name@champion
set_registered_model_alias
```

Một phiên bản có thể mang nhiều bí danh, nên `champion` và `challenger` cùng tồn tại để chạy A/B. Môi trường (dev, staging, prod) tách thành các mô hình đăng ký riêng, chuyển bằng `copy_model_version`.

Nguồn: ML flow docs, Migrating from Stages (v3.16.1): giai đoạn bị đánh dấu deprecated từ 2.9.0 và vẫn còn trong 3.16.1

Kho mô hình: API và thực hành tốt

```python
import mlflow
from mlflow import MlflowClient

client = MlflowClient()
mv = mlflow.register_model(
    f"runs:/{run_id}/model", "sentiment_classifier")

# eval gate passed -> tag, then point the alias
client.set_model_version_tag(
    "sentiment_classifier", mv.version,
    "validation_status", "passed")
client.set_registered_model_alias(
    "sentiment_classifier", "challenger", mv.version)

# serving code loads by alias, never by number
model = mlflow.pyfunc.load_model(
    "models:/sentiment_classifier@champion")
```

Đặt tên `{tác_vụ}_{kiến_trúc}`, ví dụ `fraud_lgbm`.

Thẻ Ghi `data_version`, `git_commit`, `eval_accuracy` để truy ngược khi có sự cố.

Phê duyệt Ít nhất một người duyệt trước khi chuyển `challenger` thành `champion`.

Hoàn tác Gán lại `champion` cho phiên bản cũ: một lệnh gọi API, không cần triển khai lại.

Đánh giá ứng dụng LLM bằng mlflow.genai.evaluate


```python
import mlflow
from mlflow.genai import scorer
from mlflow.genai.scorers import Correctness, Guidelines

@scorer
def is_concise(outputs: str) -> bool:
    return len(outputs.split()) <= 60

results = mlflow.genai.evaluate(
    data=eval_dataset,   # [{"inputs":..., "expectations":...}]
    predict_fn=qa_predict_fn,
    scorers=[
        Correctness(),
        Guidelines(name="is_vietnamese",
                   guidelines="Answer in Vietnamese"),
        is_concise,
    ],
)
```

Bộ chấm điểm dựng sẵn (LLM-as-judge: dùng mô hình ngôn ngữ để chấm điểm đầu ra): `Correctness`, `Safety`, `Guidelines`, `RelevanceToQuery`, `RetrievalGroundedness`, `ToolCallCorrectness`.

Bộ chấm điểm tự viết là hàm Python gắn `@scorer`.

Kho lời nhắc có phiên bản bất biến và bí danh: `prompts:/name@production`. Đổi bí danh là đổi lời nhắc đang chạy, có lịch sử.

Mô hình chấm điểm mặc định là `gpt-4o-mini`; có thể đổi sang nhà cung cấp khác.

Kiểm thử hồi quy cho agent bằng pytest

```python
# pyproject.toml:
#   addopts = ["-p", "mlflow.pytest.plugin"]
import mlflow
from mlflow.genai.scorers import Guidelines

@mlflow.test
def test_support_hours_in_vietnamese(agent):
    result = mlflow.genai.evaluate(
        predict_fn=agent.invoke,
        data=[{"inputs": {
            "question": "Gio ho tro cua cong ty?"}}],
        scorers=[Guidelines(
            name="is_vietnamese",
            guidelines="Answer must be in Vietnamese")],
    )
    assert result.passed, result.reason
```

Tài liệu MLflow phân biệt hai việc:

- **Đo lường**: điểm trung bình trên cả tập dữ liệu
- **Chặn**: một hành vi cụ thể có bị hỏng không

Mỗi hành vi quan trọng là một hàm kiểm thử; CI chạy như mọi bộ kiểm thử khác và thất bại khi `result.passed` là sai.

Chi tiết cổng đánh giá trong CI: xem chương 4.

DVC: phiên bản dữ liệu và chuỗi xử lý

Git cho dữ liệu: con trỏ, bộ nhớ đệm theo nội dung, kho từ xa và chuỗi xử lý tái lập

Vấn đề: dữ liệu không có kiểm soát phiên bản

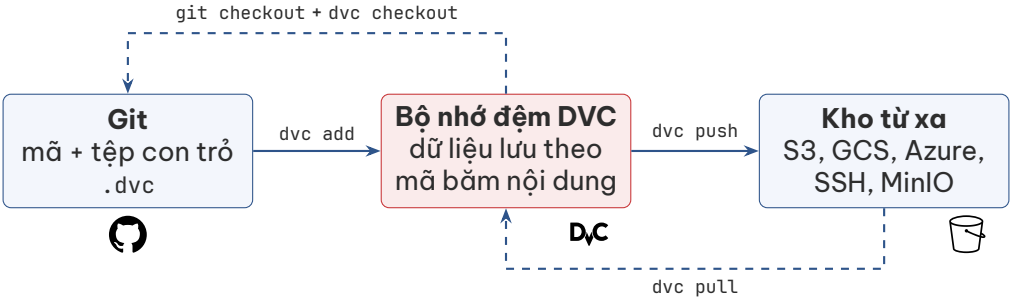
Không tái tạo được kết quả
Hôm qua độ chính xác 92%, hôm nay 88%: do mã hay do dữ liệu đã đổi?

Git không chứa nổi tệp lớn
Tệp hàng GB đặt trong thư mục chung: ghi đè lẫn nhau, không có lịch sử.

Thí nghiệm không gắn với dữ liệu
MLflow ghi chỉ số nhưng không biết huấn luyện trên phiên bản dữ liệu nào, nên so sánh vô nghĩa.

Lãng phí lưu trữ
Mỗi người sao chép dữ liệu riêng, nhiều bản trùng lặp, không loại trùng.

DVC hoạt động thế nào



- **Tệp con trỏ:** tệp nhỏ lưu mã băm của dữ liệu. Git theo dõi tệp này, nên chuyển nhánh mã cũng chuyển đúng phiên bản dữ liệu.
- **Lưu theo nội dung:** hai phiên bản giống nhau chỉ lưu một bản; làm việc cục bộ không cần mạng.

DVC: quy trình làm việc và kho từ xa

```
git init && dvc init
dvc add data/training_set.parquet
git add data/training_set.parquet.dvc data/.gitignore
git commit -m "track training dataset v1"

# remote storage (one default)
dvc remote add -d store s3://my-bucket/dvc-store
dvc push

# teammate or CI runner
git clone <repo> && dvc pull
```

Nguồn: DVC FAQ (18/11/2025); tài liệu DVC

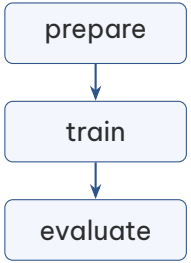
Thực hành tốt cho kho từ xa

- Dùng S3 hoặc GCS cho nhóm: có phiên bản và kiểm soát truy cập
- CI dùng vai trò quản lý danh tính và quyền truy cập (IAM) qua OIDC, không dùng khoá truy cập cố định
- Lưu thông tin xác thực cục bộ bằng `dvc remote modify --local`
- Chính sách vòng đời: chuyển dữ liệu nguội sang lưu trữ rẻ

Từ 11/2025, DVC thuộc lakeFS, vẫn giấy phép Apache-2.0.

Chuỗi xử lý tái lập với dvc.yaml

```yaml
# dvc.yaml
stages:
  prepare:
    cmd: python src/prepare.py
    deps: [data/raw, src/prepare.py]
    outs: [data/processed]
  train:
    cmd: python src/train.py
    deps: [data/processed, src/train.py]
    params: [lr, epochs]
    outs: [models/model.pkl]
    metrics:
      - metrics.json: {cache: false}
  evaluate:
    cmd: python src/evaluate.py
    deps: [models/model.pkl, data/test]
    metrics:
      - eval.json: {cache: false}
```



`dvc repro` chạy lại chỉ các giai đoạn có đầu vào thay đổi

`dvc dag` vẽ đồ thị phụ thuộc

`dvc metrics diff` so sánh chỉ số giữa hai commit

`dvc params diff` so sánh tham số

Thí nghiệm với DVC và so sánh các công cụ

```
# grid search: comma = each value
dvc exp run --queue \
  -S lr=0.001,0.01,0.1 -S epochs=10,20
dvc queue start --jobs 3

dvc exp show        # compare in table
dvc exp apply <exp> # take the best
dvc exp branch <exp> best-lr
```

`foreach` và `matrix` trong `dvc.yaml` tạo nhiều giai đoạn từ một khuôn.

	DVC	MLflow
Trọng tâm	Phiên bản dữ liệu, chuỗi xử lý	Chỉ số, kho mô hình, truy vết LLM
Đơn vị	Tệp, giai đoạn	Lần chạy, mô hình, vết
Lưu ở	Git + kho từ xa	Máy chủ theo dõi
Dùng khi	Tái lập dữ liệu	So sánh và quản lý mô hình

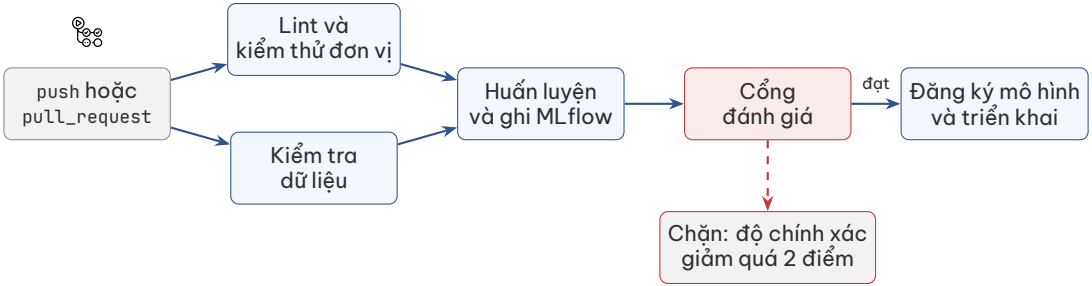
Dùng cả hai: `git commit` ghi phiên bản mã, `dvc push` ghi phiên bản dữ liệu, MLflow ghi kết quả và liên kết hai phần qua thẻ `data_version`.

Với dữ liệu quy mô doanh nghiệp, lakeFS là hướng thay thế.

GitHub Actions: CI và cổng đánh giá

Chuỗi CI cho hệ thống AI: kiểm tra dữ liệu, huấn luyện, cổng đánh giá, bảo mật luồng công việc

Kiến trúc chuỗi CI cho dự án AI



Bộ lọc đường dẫn (path filter): bước huấn luyện chỉ chạy khi `data/`, `src/` hoặc `params.yaml` thay đổi; sửa tài liệu không tốn GPU.

Dừng sớm (fail fast): kiểm tra dữ liệu lỗi thì dừng ngay, không chạy huấn luyện.

Chạy song song: lint, kiểm thử đơn vị và kiểm tra dữ liệu là các công việc (job) độc lập.

Cổng đánh giá: so mô hình mới với mô hình đang chạy; chặn triển khai nếu suy giảm vượt ngưỡng.

Luồng công việc GitHub Actions: cấu trúc

```yaml
name: ai-cicd
on:
  pull_request: {branches: [main]}
  push:         {branches: [main]}
permissions: {contents: read}   # least privilege

jobs:
  data-validation:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7        # pin to SHA in prod
      - uses: actions/setup-python@v7
        with: {python-version: "3.12", cache: pip}
      - run: pip install -r requirements.txt && dvc pull data/
      - run: python ci/validate_data.py   # fail fast
  train:
    needs: [data-validation]
    runs-on: ubuntu-latest
    steps: [...]
```

`permissions` Quyền mặc định tối thiểu; từng công việc xin thêm khi cần.

`needs` Huấn luyện chỉ chạy khi kiểm tra dữ liệu đạt.

`cache: pip` Lưu bộ nhớ đệm thư viện, rút ngắn bước cài đặt.

Phiên bản tại 10/2026: checkout v7, setup-python v7; đổi nhanh, cần kiểm tra lại.

Bảo mật luồng công việc: vì sao phải ghim phiên bản

VINUNIVERSITY

Sự cố tj-actions/changed-files (03/2025)

- Mã độc được gắn vào nhiều thẻ phiên bản cũ của hành động; hơn 23.000 kho mã bị ảnh hưởng (CVE-2025-30066)
- Mã độc đưa thông tin bí mật (secret) của CI/CD vào nhật ký chạy
- Kho dùng thẻ như `@v45` lấy ngay mã độc, không đổi dòng nào trong luồng công việc

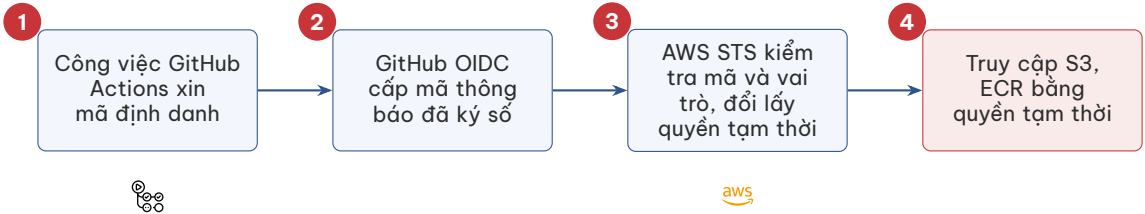
Biện pháp

1. Ghim hành động bên thứ ba vào **mã băm commit đầy đủ**; đây là cách duy nhất để dùng một hành động như bản phát hành bất biến theo tài liệu GitHub
2. Bật chính sách bắt buộc ghim SHA; dùng Dependabot để cập nhật
3. `permissions` tối thiểu cho `GITHUB_TOKEN`
4. Dùng OIDC thay khoá đám mây dài hạn
5. Môi trường (Environment) có người duyệt bắt buộc cho bước triển khai vận hành

Ví dụ: `uses: actions/checkout@<40 ký tự SHA> # v7.0.1`

Nguồn: GitHub Advisory GHSA-mrrh-fwg8-r2c3; GitHub Docs, Secure use reference

OIDC: CI lấy quyền tạm thời, không giữ khoá dài hạn



- Khoá chỉ hợp lệ trong một công việc; GitHub Secrets không chứa khoá dài hạn.
- Cần `permissions: id-token: write` và `aws-actions/configure-aws-credentials@v6` với `role-to-assume`; vai trò IAM giới hạn theo kho mã và nhánh.

Kiểm tra dữ liệu và bước huấn luyện

```yaml
on:
  pull_request:
    paths: ["data/**", "src/**", "params.yaml",
            "dvc.yaml", "*.dvc"]

  train:
    needs: [data-validation]
    runs-on: ubuntu-latest
    permissions: {id-token: write, contents: read}
    steps:
      - uses: actions/checkout@v7
      - uses: aws-actions/configure-aws-credentials@v6
        with:
          role-to-assume: ${{ vars.CI_ROLE_ARN }}
          aws-region: ap-southeast-1
      - run: dvc pull && dvc repro
      - run: python ci/log_to_mlflow.py
        env:
          MLFLOW_TRACKING_URI: ${{ secrets.MLFLOW_URI }}
```

Các kiểm tra chất lượng dữ liệu

- Lược đồ: đúng cột, đúng kiểu
- Tỉ lệ rỗng mỗi cột $\leq 5\%$
- Khoảng giá trị: tuổi 0–120, giá > 0
- Trôi phân phối so với mốc chuẩn
- Hàng trùng lặp $< 0{,}1\%$
- Cân bằng nhãn: lớp thiểu số $\geq 10\%$
- Khối lượng: tối thiểu 10.000 hàng mỗi lô
- Độ mới: dữ liệu không quá 7 ngày

Các ngưỡng là ví dụ; đặt theo từng bài toán. Công cụ: Great Expectations, Pandera.

Cổng đánh giá: lớp bảo vệ quan trọng nhất

```python
# ci/compare_models.py
import sys, mlflow

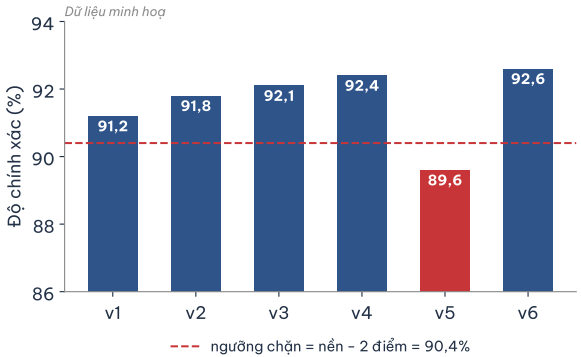
def eval_gate(new_uri, base_uri, threshold=0.02):
    new = mlflow.pyfunc.load_model(new_uri)
    base = mlflow.pyfunc.load_model(base_uri)
    new_acc = evaluate(new, X_test, y_test)
    base_acc = evaluate(base, X_test, y_test)
    delta = new_acc - base_acc
    print(f"new={new_acc:.4f} base={base_acc:.4f} "
          f"delta={delta:+.4f}")
    if delta < -threshold:
        print("FAIL: regression beyond threshold")
        sys.exit(1)        # job fails -> deploy blocked
    print("PASS")
```

Thực hành tốt

- Dùng tập kiểm tra **cố định**, không xáo trộn mỗi lần
- Theo dõi nhiều chỉ số: độ chính xác, F1, AUC, độ trễ p95
- Ngưỡng riêng cho từng chỉ số: ví dụ độ chính xác -2 điểm, độ trễ $+10\%$
- Ghi kết quả vào MLflow và đăng lên bình luận của pull request
- Kết quả sát ngưỡng: yêu cầu người duyệt thay vì tự động triển khai

Gọi trong luồng công việc: `python ci/compare_models.py --new models:/sentiment_classifier@challenger --base models:/sentiment_classifier@champion`

Cổng đánh giá trong thực tế: ngưỡng chặn và phiên bản bị loại



Phiên bản v5 có độ chính xác 89,6%, thấp hơn mốc đang chạy (92,4%) hơn 2 điểm, nên cổng chặn và v5 không được đăng ký.

Phiên bản v6 phục hồi lên 92,6% và đi tiếp.

Lưu ý: cổng chỉ chặn được điều mà nó đo. Sự cố GPT-4o 04/2025 cho thấy bộ đánh giá đạt vẫn chưa đủ; cần bổ sung chỉ số mới khi phát hiện kiểu lỗi mới.

Cổng đánh giá cho ứng dụng LLM và agent

```
# promptfoo: non-zero exit if any test fails
npx promptfoo@latest eval --fail-on-error

# DeepEval: pytest wrapper, metric thresholds
deepeval test run tests/test_agent.py

# MLflow: pytest + scorers
pytest tests/ -p mlflow.pytest.plugin
```



MLflow



pytest



Actions

Mẫu chung: bộ ca kiểm thử vàng (golden dataset) + chỉ số + ngưỡng + mã thoát khác 0 trên pull request.

Điểm cần lưu ý

- Ngưỡng trong tài liệu công cụ (ví dụ 95%) chỉ là ví dụ, không phải chuẩn
- Mô hình chấm điểm có thiên lệch vị trí, độ dài, tự ưu ái; hợp lý hoá bằng cố định phiên bản mô hình chấm, nhiệt độ thấp, chạy lặp
- Với agent, đánh giá cả quỹ đạo gọi công cụ, không chỉ câu trả lời cuối

OpenAI công bố mua lại promptfoo (03/2026); bản mã nguồn mở vẫn giữ giấy phép MIT.

Nguồn: Tài liệu promptfoo, DeepEval, MLflow; Zheng và cộng sự, arXiv:2306.05685

Thực hành tốt cho CI của kho mã AI

Bí mật và danh tính
Dùng OIDC cho đám mây; GitHub Secrets cho khoá còn lại; ghim SHA cho hành động bên thứ ba.

Bộ nhớ đệm
Đệm pip theo mã băm của `requirements.txt`; đệm DVC để không tải lại dữ liệu.

Ma trận phiên bản
Kiểm thử trên nhiều phiên bản Python; với AI có thể thêm kích thước mô hình, phiên bản CUDA.

GPU trong CI
Runner GPU T4 (16 GB VRAM) của GitHub: khoảng 0,052 USD mỗi phút; A100, H100 cần runner tự quản lý.

Chạy có điều kiện
Bộ lọc đường dẫn và `concurrency`: pull request chỉ sửa tài liệu bỏ qua huấn luyện.

Thông báo
Gửi Slack khi chuỗi lỗi hoặc cổng chặn; đăng kết quả đánh giá vào pull request.

Nguồn: GitHub Docs, bảng giá runner (2026); GitHub Changelog (08/07/2024)

CD: chiến lược triển khai và GitOps

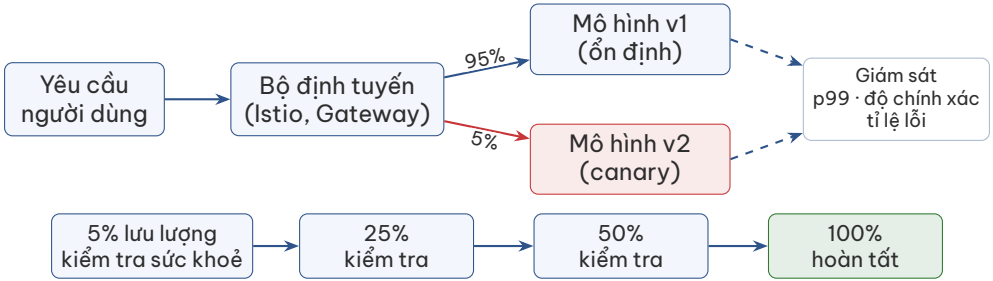
Giảm rủi ro khi đưa mô hình mới vào vận hành: canary, blue/green, shadow và điều kiện tự động hoàn tác

05

Bốn chiến lược triển khai mô hình

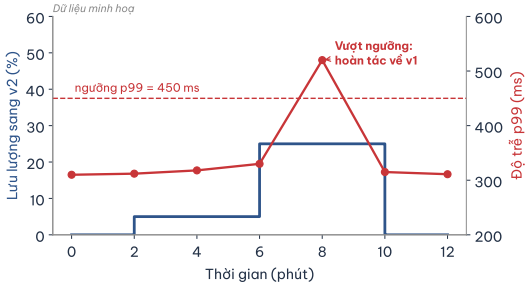
Chiến lược	Cách hoạt động	Ưu điểm	Hạn chế
Canary (triển khai thăm dò từng phần)	Chuyển 5% lưu lượng truy cập sang mô hình mới, tăng dần khi chỉ số ổn	Phát hiện lỗi sớm, rủi ro tăng từng bước	Cần giám sát tốt; logic chia lưu lượng phức tạp
Blue/green (hai môi trường song song)	Dựng v2 song song v1, đổi bộ cân bằng tải khi sẵn sàng	Không gián đoạn, hoàn tác tức thì	Tốn gấp đôi tài nguyên khi chạy cả hai
Shadow (chạy bóng)	Mô hình mới nhận bản sao yêu cầu, phản hồi chỉ được ghi lại	Thử với lưu lượng thật, người dùng không bị ảnh hưởng	Tốn gấp đôi tính toán; không thấy phản ứng người dùng
Rolling (cập nhật cuốn chiếu)	Thay từng pod trong Kubernetes	Đơn giản, là mặc định	Hoàn tác chậm; hai phiên bản cùng chạy

Canary: chia lưu lượng và điều kiện hoàn tác



Điều kiện hoàn tác: độ trễ p99 vượt ngưỡng **hoặc** độ chính xác giảm quá 2 điểm thì tự động đưa lưu lượng về v1.

Canary tự động: Argo Rollouts và Prometheus



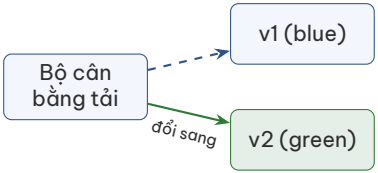

```yaml
kind: AnalysisTemplate
metadata: {name: p99-latency}
spec:
  metrics:
  - name: p99
    interval: 1m
    failureLimit: 1
    successCondition: result[0] < 0.45
    provider:
      prometheus:
        address: http://prometheus:9090
        query: |
          histogram_quantile(0.99, sum(rate(
            request_seconds_bucket{track="canary"}
            [2m])) by (le))
```

Khi điều kiện thất bại, Argo Rollouts huỷ lần phát hành và đặt trọng số canary về 0. Với mô hình, thay truy vấn bằng chỉ số chất lượng. Truy vấn trên là ví dụ tự soạn.

Nguồn: Argo Rollouts docs (bản 1.10, 08/2026); đồ thị là dữ liệu minh hoạ

Blue/green và shadow: hai cách giảm rủi ro khác nhau

Blue/green

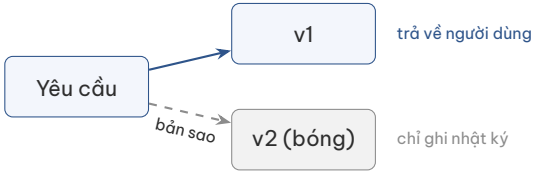


Chạy kiểm thử khói (smoke test) và kiểm thử tích hợp trên v2, rồi đổi bộ cân bằng tải. Giữ v1 thêm một khoảng thời gian để hoàn tác tức thì.

Dùng khi: bản phát hành lớn, cần không gián đoạn.

Nguồn: Istio docs, Mirroring

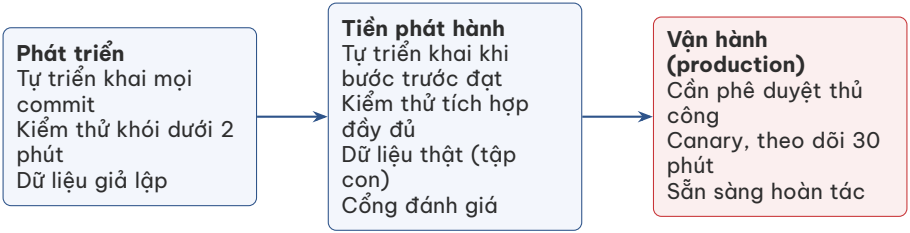
Shadow



Istio `mirror` gửi bản sao yêu cầu theo kiểu "gửi rồi bỏ qua": phản hồi của v2 bị loại. So sánh đầu ra, độ trễ, lỗi giữa hai phiên bản.

Dùng khi: thay đổi lớn, muốn thử trên lưu lượng thật trước.

Đa môi trường và GitOps với Argo CD





GitOps: trạng thái mong muốn của cụm nằm trong Git; Argo CD đồng bộ tự động từ kho mã, và hoàn tác bằng `git revert`. Mô hình được tham chiếu bằng phiên bản trong tệp khai báo, nên mọi thay đổi có dấu vết và người duyệt.

Công cụ phát hành và phục vụ trên Kubernetes

Công cụ	Phiên bản (10/2026)	Vai trò
Argo CD	v3.5	GitOps: đồng bộ cụm theo Git
Argo Rollouts	v1.10	Canary, blue/green, phân tích qua Prometheus
Flagger	v1.45	Canary, A/B, mirroring (Istio, Linkerd, Gateway API)
KServe	v0.21	Phục vụ mô hình; canary chỉ ở chế độ serverless
Gateway API Inference Extension	v1.6	Định tuyến cho LLM tự lưu trữ (`InferencePool`)
llm-d, vLLM production-stack	v0.10; Helm 0.1.x	Phục vụ LLM phân tán trên Kubernetes


Argo


Kubernetes


Istio


Prometheus


vLLM



Helm

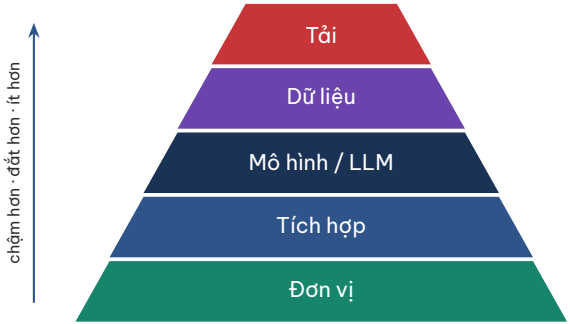
Nguồn: GitHub Releases API (06/10/2026); tài liệu KServe, Argo, Flagger

06

Kim tự tháp kiểm thử cho hệ thống AI

Từ kiểm thử đơn vị đến kiểm thử tải: mỗi tầng bắt một loại lỗi khác nhau

Kim tự tháp kiểm thử cho hệ thống AI



k6, Locust: p95 dưới 500 ms tại 50 RPS

Great Expectations, Pandera: lược đồ, phân phối, chất lượng

pytest, MLflow, promptfoo: hành vi, hồi quy, an toàn

pytest: chuỗi suy luận đầu-cuối với mẫu nhỏ

pytest: tiền xử lý, tách từ, kỹ thuật đặc trưng

Tầng thấp chạy nhanh, nhiều, trên mọi commit; tầng cao chạy ít hơn, ở bước phát hành hoặc theo lịch.

Kiểm thử đơn vị và tích hợp

```python
from src.preprocessing import clean_text, tokenize

def test_clean_text_removes_html():
    assert clean_text("<b>hello</b>") == "hello"

def test_tokenize_respects_max_len():
    assert len(tokenize("word " * 600, max_len=512)) <= 512

def test_features_have_no_label_leakage():
    feats = engineer_features(make_test_df(n=100),
                              target_col="label")
    assert "label" not in feats.columns

def test_inference_e2e(client):
    r = client.post("/predict", json={"text": "Great!"})
    assert r.status_code == 200
    assert 0.0 <= r.json()["confidence"] <= 1.0
```

Danh mục kiểm tra

- Từng bước tiền xử lý, riêng lẻ
- Ca biên: đầu vào rỗng, null, độ dài tối đa
- Ký tự đặc biệt và tiếng Việt có dấu
- Đặc trưng không rò rỉ nhãn (data leakage)
- Giả lập (mock) lời gọi API và cơ sở dữ liệu
- Độ bao phủ $\geq$ 80% cho `src/`
- Toàn bộ kiểm thử đơn vị dưới 30 giây
- Kiểm thử tích hợp dùng mẫu nhỏ (100 mẫu)

Kiểm thử mô hình, dữ liệu và LLM

VINUNIVERSITY

Mô hình (hành vi)

- **Bất biến:** xoay ảnh 90° thì nhãn không đổi
- **Có hướng:** thêm "không" vào câu thì cảm xúc đảo chiều
- **Hồi quy:** độ chính xác trên tập vàng $\geq$ bản trước $-$ 0,5 điểm
- **Công bằng:** chênh lệch độ chính xác giữa nhóm con dưới 3 điểm

Dữ liệu

- **Lược đồ:** đúng cột, đúng kiểu, không cột thừa
- **Đầy đủ:** tỉ lệ rỗng dưới ngưỡng
- **Phân phối:** kiểm định KS so với mốc chuẩn
- **Khối lượng:** số hàng trong khoảng kỳ vọng
- Chạy trên mỗi phiên bản dữ liệu mới, ghi kết quả vào MLflow

LLM và agent

- **Bộ ca vàng:** câu hỏi thật từ nhật ký, có đáp án chuẩn
- **Quỹ đạo:** agent gọi đúng công cụ, đúng thứ tự
- **An toàn:** từ chối nội dung độc hại; thử tấn công chèn lệnh (prompt injection)
- **Chi phí:** số token và độ trễ mỗi tác vụ không vượt ngân sách
- Chấm điểm bằng quy tắc trước, LLM-as-judge sau

Kiểm thử tải với k6

```
import http from 'k6/http';

export const options = {
  scenarios: { baseline: {
    executor: 'constant-arrival-rate',
    rate: 50, timeUnit: '1s', duration: '5m',
    preAllocatedVUs: 100,
  }},
  thresholds: {
    http_req_duration: ['p(95)<500'],
    http_req_failed: ['rate<0.001'],
  },
};

export default function () {
  http.post(`${__ENV.URL}/invocations`,
    JSON.stringify({inputs: [{text: 'Great product!'}]}),
    {headers: {'Content-Type': 'application/json'}});
}
```

Bốn loại kiểm thử tải

- **Mốc chuẩn:** 50 RPS, p95 dưới 500 ms, lỗi dưới 0,1%
- **Chịu tải (stress):** tăng dần tới 500 RPS để tìm điểm gãy
- **Bền bỉ (soak):** 50 RPS trong 1 giờ, tìm rò rỉ bộ nhớ
- **Đột biến (spike):** 0 lên 500 RPS đột ngột, kiểm tra tự động mở rộng

Ngưỡng (`thresholds`) vi phạm thì k6 thoát với mã khác 0 và chuỗi CI thất bại.

Với LLM, đo thêm thời gian đến token đầu (TTFT) và độ trễ giữa các token.

07

Phục vụ mô hình và kiểm thử A/B

Từ kho mô hình đến điểm cuối vận hành, rồi so sánh hai phiên bản bằng thống kê

Phục vụ mô hình: bốn mức

```
# 1. Local: dev and testing
mlflow models serve \
  -m "models:/sentiment_classifier@champion" \
  -p 5000 --env-manager local

curl -X POST http://localhost:5000/invocations \
  -H "Content-Type: application/json" \
  -d '{"inputs": [{"text": "Great product!"}]}'

# 2. Docker image for staging
mlflow models build-docker \
  -m "models:/sentiment_classifier@champion" \
  -n sentiment-serving:v1.0
```

Local `mlflow models serve`: khởi động nhanh, dùng khi phát triển.

Docker `build-docker`: gói portable, dùng cho tiền phát hành.

Đám mây SageMaker, Azure ML, Databricks: hạ tầng được quản lý, tự mở rộng.

Kubernetes KServe, Seldon, hoặc vLLM cho LLM: tuỳ chỉnh chính sách mở rộng.


Docker


Kubernetes



AWS


Azure

Kiểm thử A/B: chia lưu lượng và kiểm định

```python
import hashlib
from scipy.stats import chi2_contingency

def variant(user_id: str, pct_b: float = 0.10) -> str:
    # deterministic: same user always sees same variant
    h = int(hashlib.sha256(user_id.encode()).hexdigest(), 16)
    return "B" if (h % 1000) < pct_b * 1000 else "A"

# after the experiment: conversions per variant
table = [[conv_a, n_a - conv_a],
         [conv_b, n_b - conv_b]]
chi2, p_value, _, _ = chi2_contingency(table)
if p_value < 0.05:
    print("significant at 95%")
```

Chia lưu lượng Băm `user_id`: mỗi người luôn thấy cùng một biến thể.

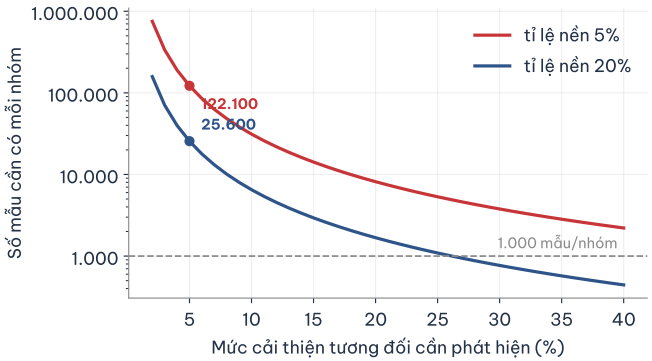
Chỉ số chính Chỉ số kinh doanh (tỉ lệ nhấp, doanh thu).

Chỉ số rào chắn Độ trễ p95, tỉ lệ lỗi, chi phí token không được xấu đi.

Mức ý nghĩa $p < 0{,}05$; nhiều so sánh cùng lúc thì hiệu chỉnh (Bonferroni).

Ghi kết quả của từng biến thể vào hệ thống phân tích hoặc MLflow, không ghi trong đường xử lý yêu cầu.

Cần bao nhiêu mẫu cho một phép thử A/B?



Quy tắc "tối thiểu 1.000 mẫu mỗi biến thể" không đủ trong phần lớn trường hợp.

Với tỉ lệ nền 5% và muốn phát hiện mức cải thiện tương đối 5%, cần khoảng **122.000** mẫu mỗi nhóm; với tỉ lệ nền 20%, khoảng **25.600**.

Tính trước bằng phân tích công suất (power analysis); không dừng sớm khi *p* vừa xuống dưới 0,05.

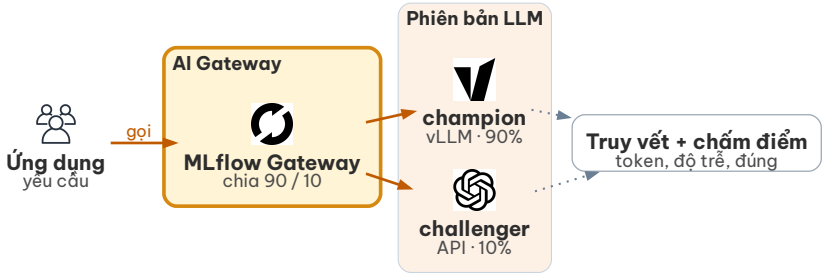
Tính theo công thức hai tỉ lệ, mức ý nghĩa 5% hai phía, công suất 80%.

A/B cho LLM: chia lưu lượng ở cổng và theo dõi chi phí

Nhóm chỉ số	Ví dụ	Cách đo
Chỉ số chính	Tỉ lệ hoàn thành tác vụ, tỉ lệ người dùng chấp nhận câu trả lời	Sự kiện sản phẩm; đánh giá có nhãn người
Chất lượng tự động	Điểm đúng, độ bám nguồn, an toàn	Bộ chấm điểm LLM-as-judge chạy trên mẫu lưu lượng
Rào chắn vận hành	Độ trễ p95, tỉ lệ lỗi, thời gian đến token đầu	Giám sát (Prometheus, OpenTelemetry)
Rào chắn chi phí	Token mỗi yêu cầu, chi phí mỗi tác vụ	Nhật ký của AI Gateway hoặc truy vết MLflow

MLflow AI Gateway chia lưu lượng giữa các mô hình, có chuỗi dự phòng và hạn mức chi phí. Biến thể thắng ở chỉ số chính nhưng làm chi phí tăng gấp đôi vẫn có thể bị từ chối.

A/B cho LLM: chia lưu lượng ở cổng AI

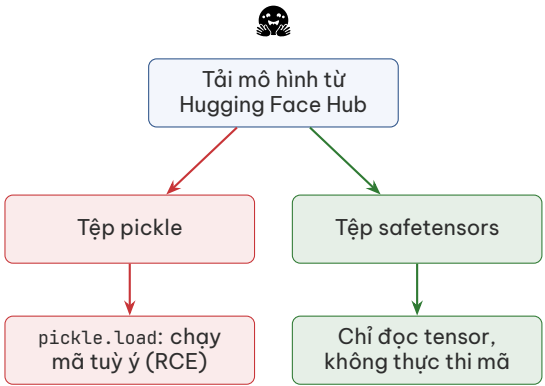


1. Ứng dụng gọi một điểm truy cập duy nhất **2.** Cổng chia 90% cho `champion`, 10% cho `challenger`
3. Mỗi lần gọi để lại vết thực thi; bộ chấm điểm và chỉ số rào chắn (độ trễ, token) so sánh hai nhánh.
Chuyển lưu lượng bằng cấu hình cổng, không cần sửa mã ứng dụng; có thể đưa nhánh `challenger` về 0% ngay khi chỉ số xấu đi.

Chuỗi cung ứng mô hình và bảo mật CI/CD

Tệp mô hình cũng là mã thực thi: kiểm soát nguồn gốc, định dạng và chữ ký trước khi triển khai

Tệp mô hình có thể chứa mã độc



Sự cố có thật: 02/2024, JFrog phát hiện khoảng 100 mô hình độc hại trên Hugging Face, một số dùng phương thức `__reduce__` của pickle để mở kết nối điều khiển từ xa (reverse shell) khi mô hình được tải.

Biện pháp

- Ưu tiên định dạng safetensors; chặn `pickle` từ nguồn không tin cậy
- Ghim mô hình theo mã băm commit, không theo tên nhánh
- Tải và quét trong môi trường cách ly trước khi đưa vào kho mô hình

Hugging Face quét pickle nhưng tài liệu của họ nói bước quét này không tuyệt đối.

Nguồn: JFrog Security Research (27/02/2024); tài liệu Hugging Face về pickle và safetensors

Ký số mô hình và chứng thực nguồn gốc

```
pip install model-signing

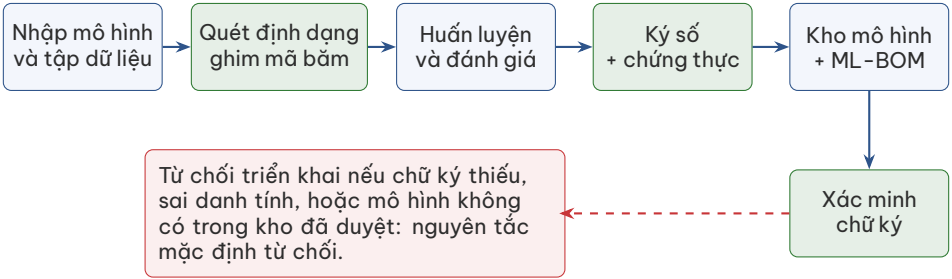
# in CI, after training: keyless sign (Sigstore)
model_signing sign ./models/sentiment

# at deploy: verify signature + who built it
model_signing verify ./models/sentiment \
  --signature model.sig \
  --identity "$CI_IDENTITY" \
  --identity_provider "$OIDC_ISSUER"

# build provenance for container images
# uses: actions/attest-build-provenance@v4
# gh attestation verify image.tar -R org/repo
```

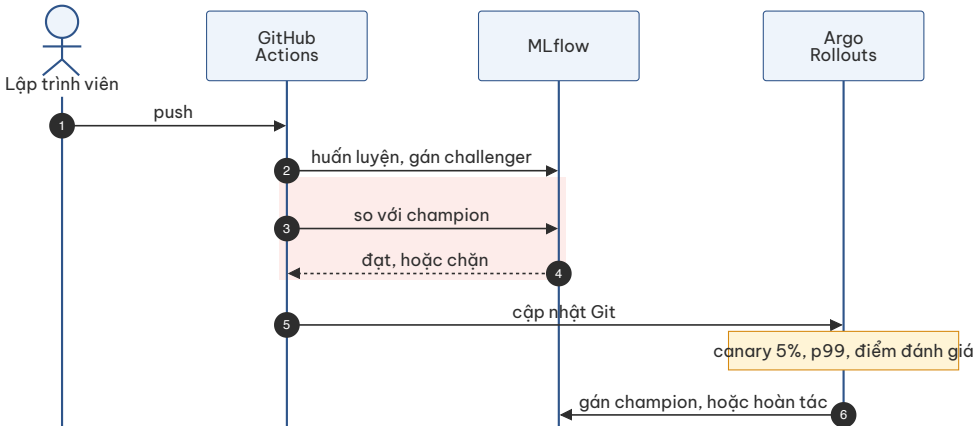
Lớp	Công cụ
Toàn vẹn, danh tính	OpenSSF model-signing (v1.0 ra mắt 04/2025), Sigstore
Nguồn gốc build	Chứng thực của GitHub, SLSA
Thành phần	ML-BOM (CycloneDX): mô hình, tập dữ liệu, phụ thuộc
Rủi ro	OWASP LLM03:2025 Supply Chain

Nguồn: Sigstore blog (04/04/2025); GitHub Docs, Artifact attestations; OWASP GenAI



Khung quản trị tham chiếu: NIST AI RMF 1.0 và hồ sơ AI sinh tạo NIST AI 600-1; OWASP Top 10 cho LLM.

Trình tự một lần phát hành mô hình



Bước 3 và 4 là cổng đánh giá; bước 6 gán `champion` cho phiên bản mới nếu canary đạt, hoặc hoàn tác về phiên bản cũ.

LIVE DEMO

1. **git push:** luồng GitHub Actions kích hoạt, các công việc chạy song song.
2. **dvc pull và dvc repro:** lấy đúng phiên bản dữ liệu, chỉ chạy lại giai đoạn thay đổi.
3. **Huấn luyện và ghi MLflow:** đăng ký phiên bản mới, gán bí danh `challenger`.
4. **Cổng đánh giá:** so với `champion`; đạt thì đi tiếp, không đạt thì chặn.
5. **Canary:** Argo Rollouts chuyển 5% lưu lượng, phân tích p99 và điểm đánh giá, rồi tăng dần.
6. **Mở rộng:** cố ý làm suy giảm mô hình; cổng chặn hoặc canary tự hoàn tác.

LAB #21

Mục tiêu: Dựng chuỗi CI/CD cho mô hình: máy chủ MLflow cục bộ (SQLite), tập dữ liệu có phiên bản DVC với chuỗi 3 giai đoạn, luồng GitHub Actions gồm kiểm thử, huấn luyện, cổng đánh giá và triển khai

Deliverable: Từ 3 thí nghiệm MLflow; `dvc repro` chạy thành công; luồng GitHub Actions đạt trên kho mã thực; mô hình được đăng ký với bí danh `challenger`

Thời gian: 2 giờ

Tổng kết — Key Takeaways

Những ý chính cần nhớ trước khi sang bài tiếp theo

1. **Tái lập cần cả mã, dữ liệu và kết quả:** Git giữ mã, DVC giữ dữ liệu, MLflow giữ thí nghiệm và mô hình. Dùng bí danh thay giai đoạn trong kho mô hình.

2. **Cổng đánh giá là lớp bảo vệ quan trọng nhất,** nhưng chỉ chặn được điều nó đo: bổ sung chỉ số mới sau mỗi kiểu lỗi mới, kể cả với LLM và agent.

3. **Phát hành từng phần:** canary với điều kiện hoàn tác tự động, hoặc shadow trước; GitOps để mọi thay đổi có dấu vết.

4. **Bảo mật chuỗi CI/CD:** ghim hành động theo SHA, OIDC thay khoá dài hạn, dùng safetensors, ký số và xác minh mô hình khi triển khai.

Bài tiếp theo

LLMOps và quản lý phiên bản prompt (Ngày 22)

"Prompt thay đổi thì hành vi thay đổi: cần truy vết, đánh số phiên bản và đánh giá mọi thay đổi"

Bài tập về nhà

- Hoàn thành Lab 21: MLflow, DVC, GitHub Actions
- Tạo tài khoản LangSmith (gói miễn phí)
- Đọc trước tài liệu truy vết và Prompt Hub của LangSmith

Hỏi & Đáp



Cảm ơn!

AICB-P2T2 · Ngày 21 · CI/CD cho hệ thống AI

Giảng viên Nguyễn Hải Dương · VinUniversity